

2.7 Exception Handling in Spring Boot Web MVC

Benefits of Exception Handling

- **Prevent application crashes.** → e.g., *divide by zero* or accessing a *null* object can *terminate* your *app*.
- **Provide user-friendly error responses.** → Instead of exposing *raw stack traces* or *technical messages* to the client, exception handling allows us to return *clear, meaningful*, and *safe error messages* (e.g., "Employee not found with id: 10").
- **Facilitate debugging and maintenance.** → By centralizing *exception handling* (e.g., using *@ControllerAdvice* in Spring Boot), developers can easily *log exceptions* in a consistent way.
- **Ensure consistent error handling across the application.** → Without proper handling, each *controller* or *service* might throw errors differently, leading to *inconsistent responses*.

Exception Handling in Spring Boot

- Although exceptions can be handled using traditional **try-catch** blocks, this approach often leads to *lengthy* and *scattered* error-handling code.
- For *cleaner, easier-to-maintain*, and *centralized* error management, it is recommended to use Spring's **@ExceptionHandler** annotation.
- This not only improves code *readability* but also ensures *consistent* and *user-friendly* error responses across the application.

```
@RestController
@RequestMapping("/employee")
public class EmployeeController {
    private final EmployeeService employeeService;

    @Autowired
    private ModelMapper mapper;

    @GetMapping("/{id}")
    public ResponseEntity<EmployeeDTO> getEmployeeById(@PathVariable("id") Long id) {
        return employeeService.getEmployeeById(id).map(ResponseEntity::ok)
            .orElseThrow(() -> new NoSuchElementException("Employee not Found with id: " + id));
    }

    @ExceptionHandler(NoSuchElementException.class)
    public ResponseEntity<String> NoSuchElementExceptionHandler(NoSuchElementException exception) {
        return new ResponseEntity<>("[Exception]- " + exception.getMessage(), HttpStatus.NOT_FOUND);
    }
}
```

- **@ExceptionHandler(NoSuchElementException.class)** tells Spring: "Whenever a *NoSuchElementException* happens inside this *controller*, call this method instead of letting the app crash."
- Spring automatically *catches* the *exception*, passes it to this method, and uses the method's return value as the *HTTP* response.

How can we handle Exceptions globally (for every controller):

Why we need?

- *Centralized* way to *handle errors*.
- No need to repeat *try-catch* or **@ExceptionHandler** in *every controller*.
- Ensures *consistent error responses* across the application.

Steps to Implement:

1. Create a DTO for error response.

```
@Builder
@Data
public class ApiError {
    private HttpStatus status;
    private String message;
    private LocalDateTime time;
}
```

@Builder: Simplifies *object creation* by implementing the *builder pattern*.

@Data: Generates boilerplate code for getters, setters, equals(), hashCode(), requiredArgsConstructor() and toString() methods.

We return an object of *ApiError* if any exception occurs (to get JSON response).

2. Create a Global Exception Handler.

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(NoSuchElementException.class)
    public ResponseEntity<ApiError> EmployeeNotFound(NoSuchElementException exception) {
        return new ResponseEntity<>(ApiError.builder()
            .status(HttpStatus.NOT_FOUND)
            .message(exception.getMessage())
            .time(LocalTime.now())
            .build(), HttpStatus.NOT_FOUND);
    }
}
```

- **@RestControllerAdvice** → @ControllerAdvice + @ResponseBody
- **@ControllerAdvice** → tells Spring: "This class contains global logic (exception handling, binding, attributes) that applies to *multiple controllers*."
- **@ResponseBody** → Returns result in *JSON/XML* instead of *html view*.

Internal working:

When an *exception* is thrown inside *any controller*:

1. Spring looks for a *local @ExceptionHandler* in that *controller*.
2. If not found → it checks *classes annotated* with **@ControllerAdvice** / **@RestControllerAdvice**.
3. If a *handler* for that *exception* exists → it is *executed*.
4. The *return* value becomes the *HTTP response*.

Handling input validation exceptions:

- Input **validation** failures typically trigger a **MethodArgumentNotValidException**.
- Unlike other exceptions, this **one** can contain **multiple validation errors** at once (for example, if several fields in a request DTO are invalid). Therefore, instead of returning a **single error message**, we need to **extract all validation errors**, collect them into a **list**, and then return them in a **structured response**.
- This approach ensures that the client receives a complete overview of all validation issues in a single response, rather than fixing them one by one.

```
@Builder
@Data
public class ApiError {
    private HttpStatus status;
    private String error;
    private LocalTime time;
    private List<String> message;
}

// handles validation exceptions in GlobalHandler
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ApiError> HandleInputValidationError(MethodArgumentNotValidException e) {
    List<String> errors = new LinkedList<>();
    e.getBindingResult()
        .getAllErrors()
        .forEach(err -> errors.add(err.getDefaultMessage()));
    return new ResponseEntity<>(ApiError.builder()
        .status(HttpStatus.BAD_REQUEST)
        .error(e.getClass().getName())
        .message(errors)
        .time(LocalTime.now())
        .build(), HttpStatus.BAD_REQUEST);
}
```